Sieci neuronowe - projekt

Paweł Osikowicz, Szymon Cichy, Konrad Pawlik

Wstęp

Celem projektu było stworzenie sieci neuronowej identyfikującej zdjęcia jako zrobione przez człowieka, lub wygenerowane przez AI.

Łącznie zbudowaliśmy 3 różne modele, każdy z nich oparty o konwolucyjną sieć neuronową - taki wybór podyktowany jest charakterem postawionego zadania (analiza obrazu). Rozważane modele różnią się rozmiarem, głębokością, oraz dodatkowymi warstwami liniowymi.

Opis zbioru danych i wczytanie

- Dane których użyliśmy pochodzą z konkursu organizowanego przez stronę Kaggle.
  Znajdują się w nim pary zdjęć – każde zdjęcie prawdziwe ma swój odpowiednik wygenerowany przez AI. Dodatkowo, wydzielony przez twórcę zbiór testowy nie posiadał wartości zmiennej objaśnianej, gdyż celem konkursu było znalezienie owych etykiet.
- Naturalnym rozwiązaniem było pominięcie dostarczonego zbioru testowego i wydzielenie go na nowo z zbioru treningowego. Finalnie, spośród prawie 80 000 zdjęć w zbiorze treningowym znalazło się 39 975, w zbiorze walidacyjnym 15 990, zaś w zbiorze testowym 23 985 (podział odpowiednio 50/20/30 %).
  Wczytanie naszych danych zaimplementowaliśmy przy pomocy bibliotek *zipfile* oraz *pathlib* - prawie 10GB plik ZIP pobraliśmy na dysk Google, a następnie zdjęcia wczytaliśmy wprost stamtąd.
- Po wczytaniu zdjęć, dodaniu im odpowiednich etykiet oraz obróbce (ujednolicenie rozmiaru, obrót zdjęć, zmiana jasności, normalizacja [0,1] – losowe przekształcenia wykonane tylko na zbiorze treningowym) przystąpiliśmy do budowy modeli.

Schemat uczenia modeli

- W projekcie wykorzystaliśmy możliwości metody *torch.optim.lr_scheduler* z funkcją *ReduceLROnPlateau* – pozwoliła ona nam na zmianę learning rate w trakcie uczenia w momentach braku poprawy celności modelu na zbiorze walidacyjnym.
- Postęp uczenia modeli zapisywaliśmy z pomocą metody *torch.save* – zapisywaliśmy strukturę i wagi modelu w trakcie jego uczenia co każde 5 epok w pliku pkl. Dzięki temu mogliśmy wybrać najlepszy spośród uczonych modeli, cofając się w razie przeuczenia.
- Dokładne informacje o przebiegu uczenia zapisywaliśmy w formie plików pth – pozwalały nam one na douczanie modeli jeżeli była taka potrzeba.
- Do każdego modelu wykorzystaliśmy inicjalizację Xaviera Glorota do wyboru początkowych wartości wag.
- Każdy z modeli uczyliśmy przez 15 epok oraz douczaliśmy przez kolejne 15.

Model 1

Pierwszy zbudowany przez nas model ma strukturę konwolucyjnej sieci neuronowej o dwóch warstwach konwolucyjnych, każda z funkcją aktywacji ReLU i warstwą max - pool. Wszystkie warstwy konwolucyjne mają jądro o wymiarze 3x3 z paddingiem równym 2.

Wchodzące dane dzielone są na 3 kanały RGB, a pierwsza warstwa ma 16 kanałów wyjściowych. Druga warstwa ma 32 kanały wyjściowe.

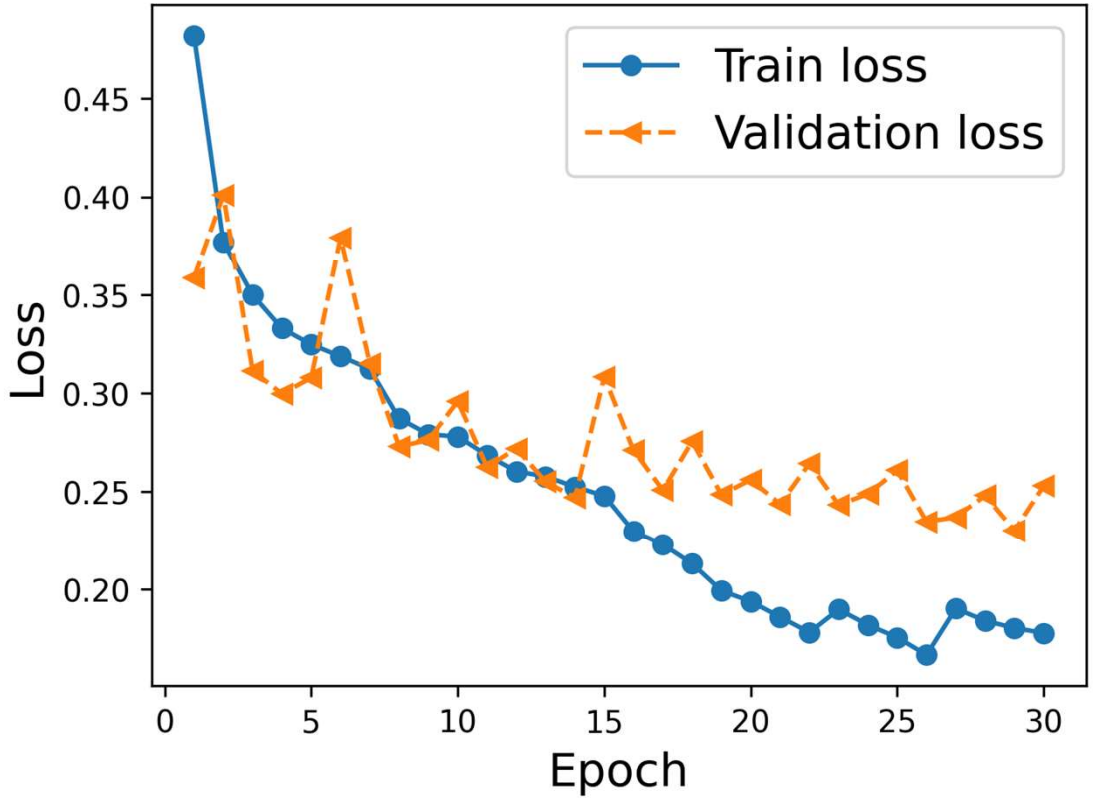
Po warstwach konwolucyjnych dokonywane jest spłaszczenie obrazu i zastosowanie warstwy liniowej w pełni połączonej z funkcją aktywacji ReLU, o liczbie neuronów 9248 i 4200 – odpowiednio wejściowych i wyjściowych.

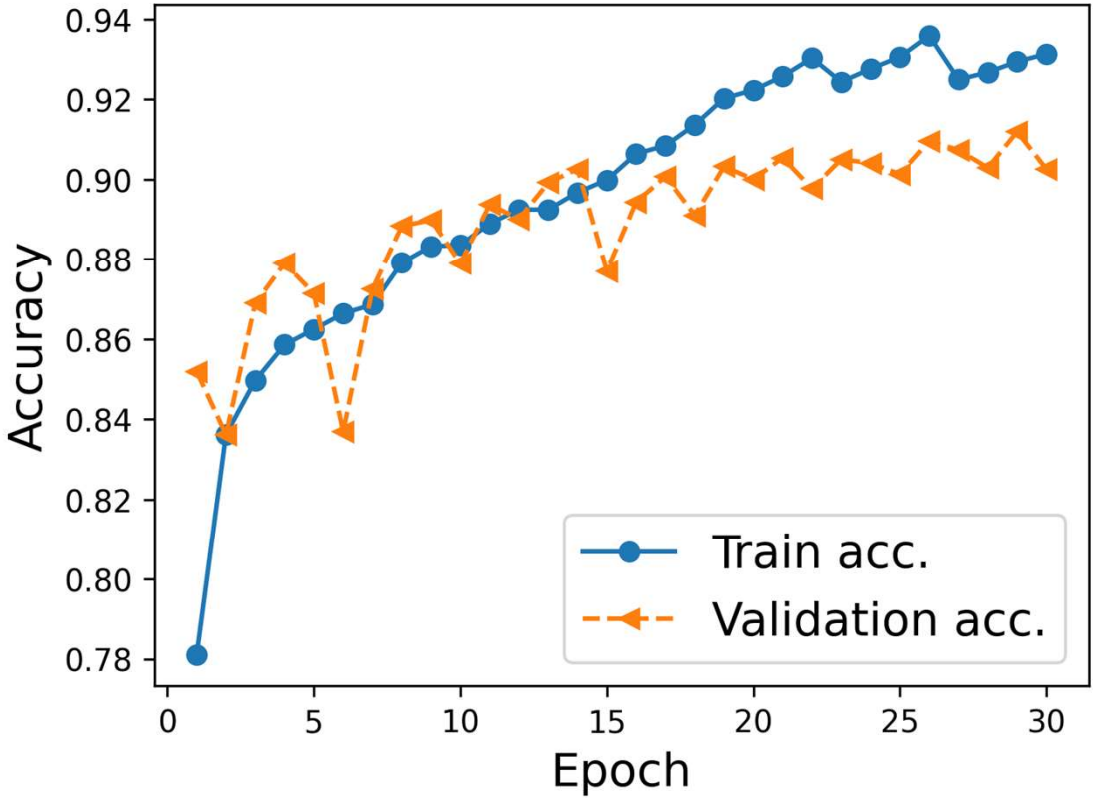
Na koniec stosujemy ostatnią warstwę w pełni połączoną, z dwoma wyjściami.

Wykorzystaliśmy technikę dropout z parametrem p=0.5 - w trakcie uczenia wyłącza ona losowo neurony z prawdopodobieństwem wyłączenia równym 0.5. Jest ona stosowana aby zapobiec przeuczeniu sieci.

Jako funkcję strat wykorzystaliśmy CrossEntropyLoss, a za optimizer wzięliśmy ADAM z współczynnikiem uczenia równym 0.0005.

Model 1 - wyniki





Model 2

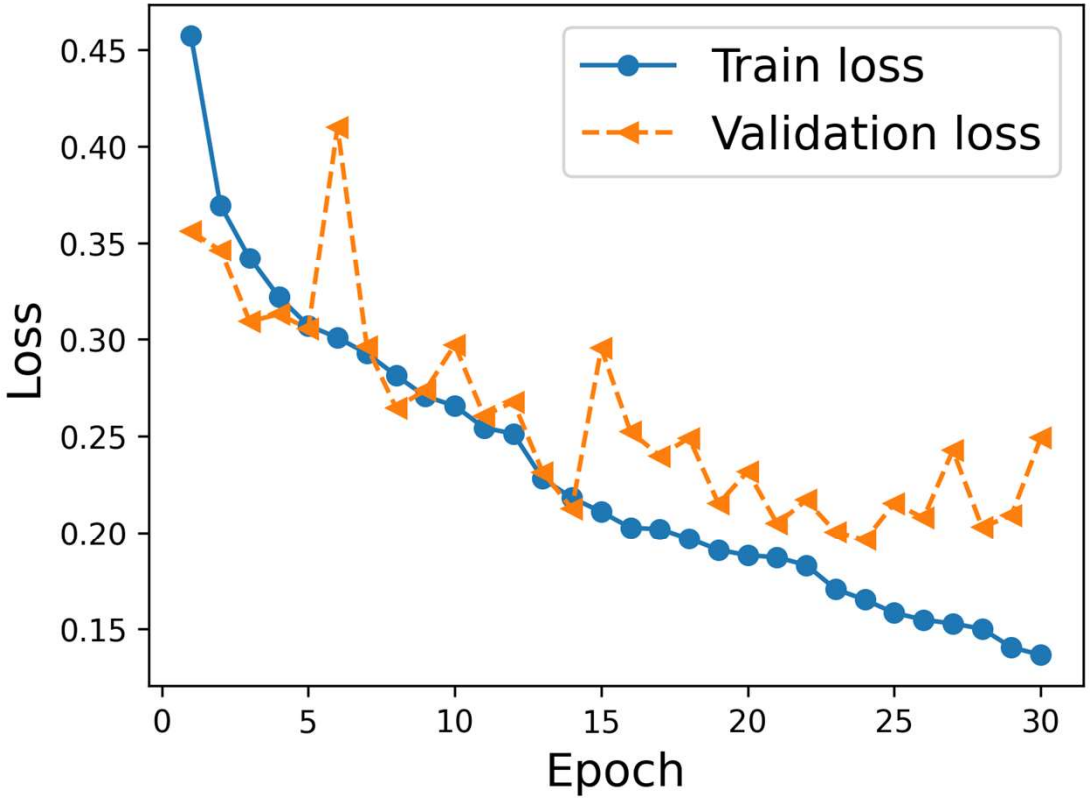
Drugi zaimplementowany model jest zdecydowanie głębszą od poprzedniej siecią. Składa się z sześciu warstw konwolucyjnych, każda z rozmiarem filtra 3x3 oraz paddingiem 2. Po każdej z warstw stosowany jest max - pooling.

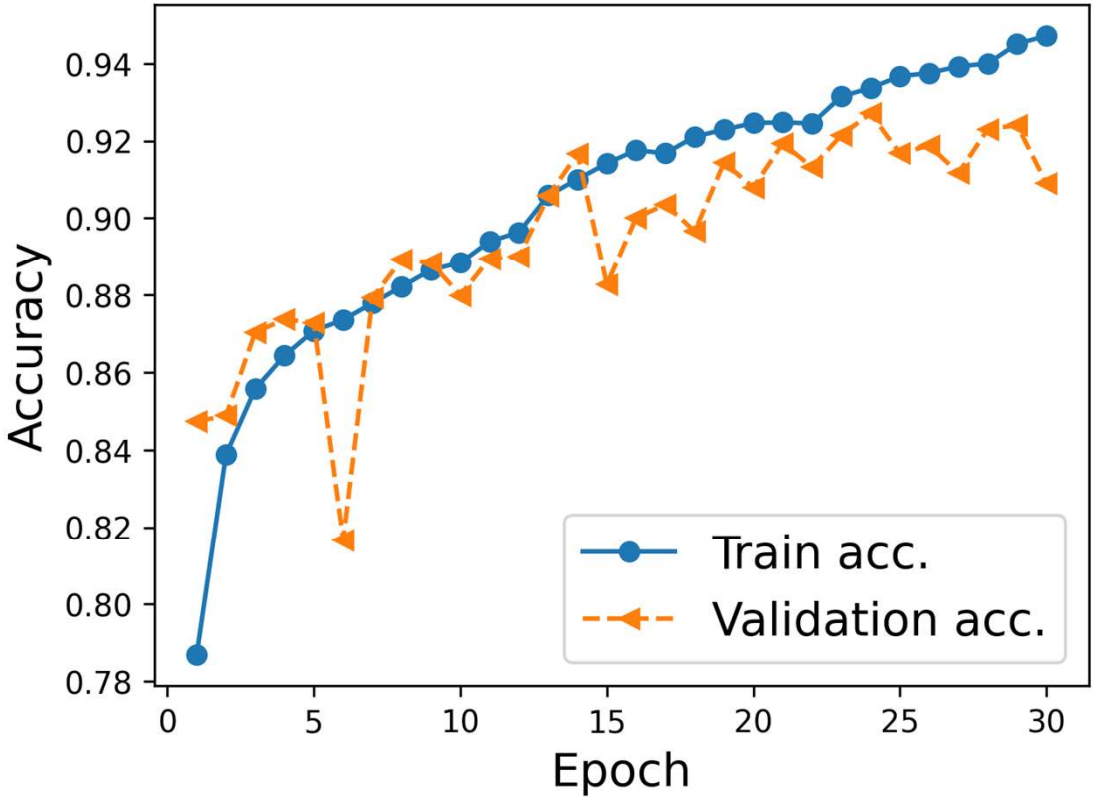
Tak jak wcześniej, wchodzące dane dzielimy na 3 kanały RGB. Wyjście z pierwszej warstwy ma rozmiar 30, z drugiej 60, z trzeciej 120, z czwartej 140, z piątej 280 i z ostatniej - 560. Po każdej warstwie konwolucyjnej stosujemy funkcję aktywacji ReLU.

Po tych sześciu warstwach spłaszczamy obraz, stosujemy warstwę w pełni połączoną o 2240/1100 wejściach i wyjściach z aktywacją ReLU i dropoutem takim jak w modelu pierwszym.

Na koniec dodaliśmy jeszcze jedną warstwę w pełni połączoną. Za funkcję strat posłużyło nam CrossEntropyLoss, za optymalizator - ADAM, z początkowym współczynnikiem uczenia 0.0002.

Model 2 - wyniki





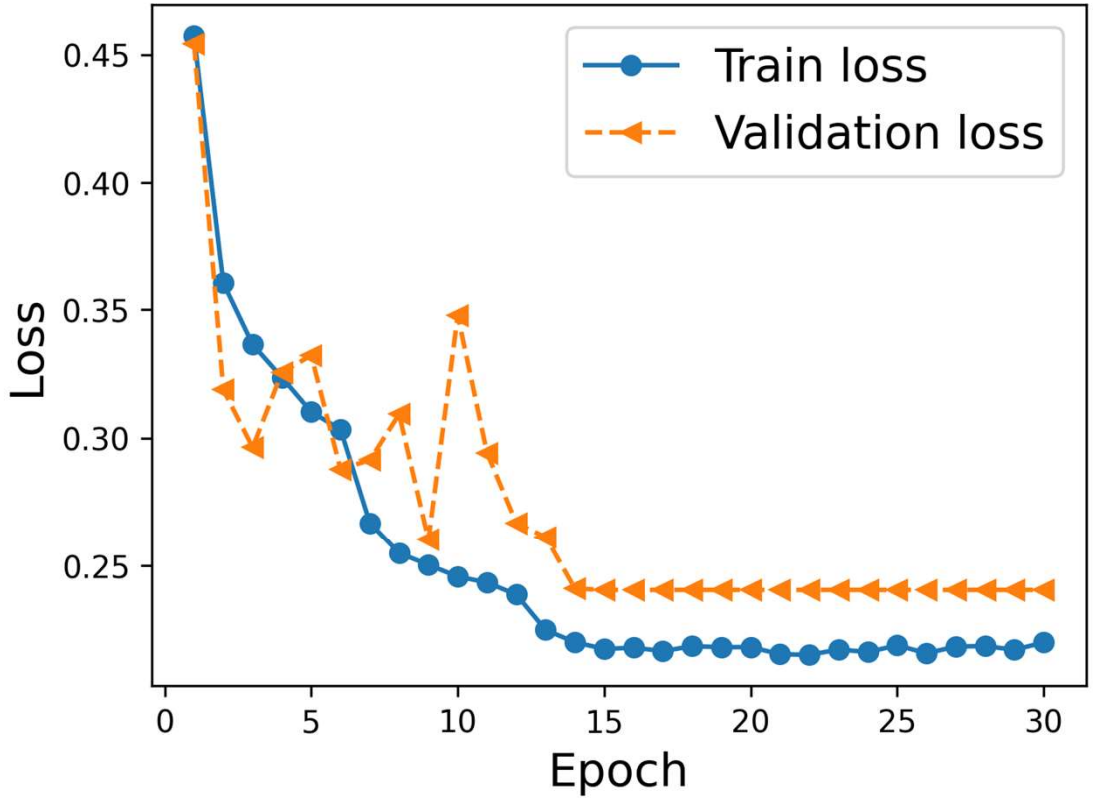
Model 3

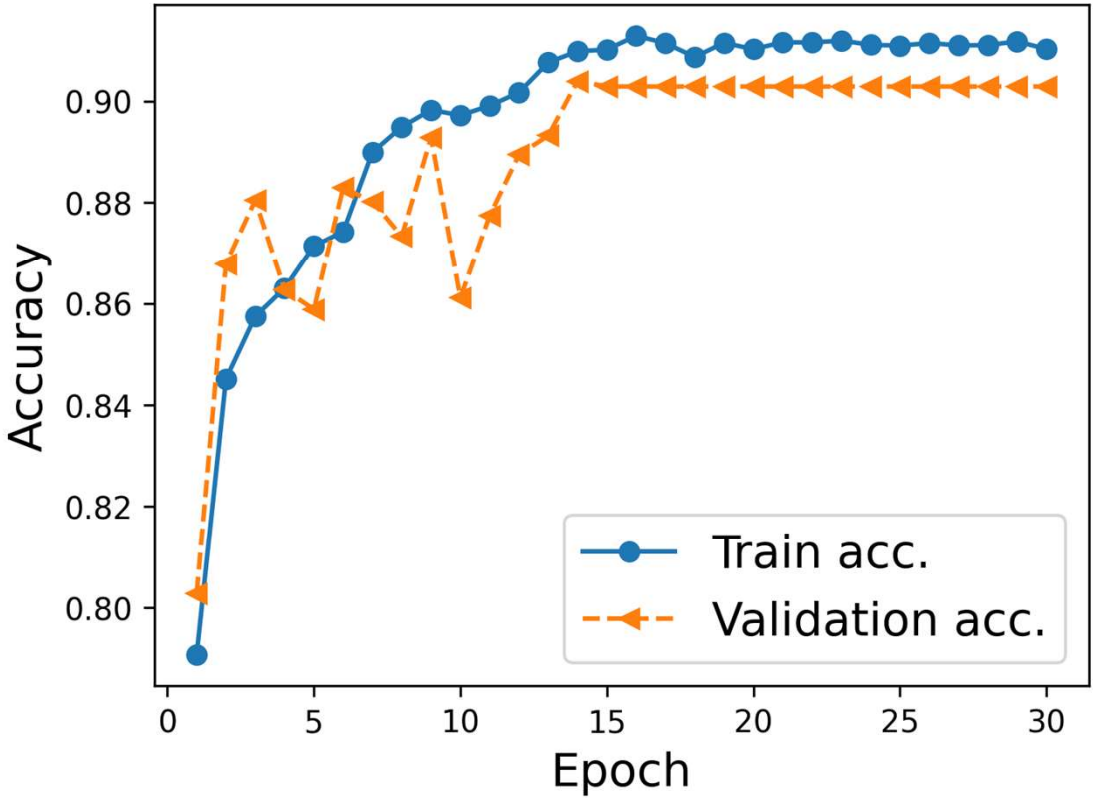
Końcowy model to model o 4 warstwach konwolucyjnych – o wyjściach kolejno 20/40/60/80. Ważnymi zmianami względem poprzednich modeli jest zmiana rozmiaru paddingu z 2 na 1 oraz zastosowanie funkcji aktywacji LeakyReLU – jest to modyfikacja klasycznej funkcji ReLu, która zamiast 0 gdy argument jest poniżej danego progu, przyjmuje wartości bliskie zeru.
Dokładny wzór: $f(x) = min\{a*x , x\}$ gdzie parametr a został ustawiony na domyślne 0.01.

Po wyjściu z warstw konwolucyjnych oraz spłaszczeniu dodaliśmy 3 warstwy liniowe – o jedną więcej niż w poprzednich. Ich wymiary to 2240/1100 – 1100/550/ - 550/2 odpowiednio wejście/wyjście. Ponownie wykorzystaliśmy funkcję LeakyReLU oraz dropout, tym razem z p = 0.3.

Funkcja strat to ponownie CrossEntropyLoss, optimizer ADAM z początkowym lr = 0.001.

Model 3 - wyniki





Wybór modelu

Najlepszy z modeli z danej architektury (odpowiednio po 15 epokach każdy) wybraliśmy wykonując predykcję na zbiorze walidacyjnym. Wyniki dla każdej z architektury to:

- Model 1: 87.7 %
- Model 2: 88.3 %
- Model 3: 90.3 %

Końcowo wybrany Model 3 na zbiorze testowym osiągnął jakość na poziomie 90.3%, porównywalny do wyniku na zbiorze walidacyjnym.

Wnioski

Postawiony cel projektu udało się rozwiązać budując dobrej jakości sieć neuronową opartą o architekturę CNN o celności ~90%. Modele tego rodzaju po dokładniejszym dopracowaniu mogłyby pomagać ludziom rozpoznawać obrazy wygenerowane przez sztuczną inteligencję, tym samym wystrzegając się dezinformacji w sieci